

Smart Defense Vision : Missile Detection and Motion Analysis on Streamlit.

¹ Mr Sunil Naidu G R, ² Chethana N, ³ Nallannagari Chandrika, ⁴ Vaishnavi V, ⁵ Vandana P

¹ Assistant Professor, Artificial Intelligence and Data Science, S J C Institute Of Technology,
Chikkaballapura(Karnataka), India

²³⁴⁵ UG Students, Artificial Intelligence and Data Science, S J C Institute Of Technology,
Chikkaballapura(Karnataka), India

Email – ¹sunilnaidugr5464@gmail.com, ²chethana960@gmail.com,

³chandhchandrika999@gmail.com, ⁴vaishuvaishali4466@gmail.com, ⁵4157vandana@gmail.com

Abstract: — The proposed work presents an AI-powered missile detection framework that uses the latest Ultralytics YOLOv11 architecture to identify missiles in both still images and video streams. The model is fine-tuned on a curated Roboflow missile dataset to improve robustness across diverse backgrounds, viewpoints, and lighting conditions, enabling reliable detection in realistic surveillance scenarios. A Python-based backend and Streamlit web interface provide secure authentication, image/video upload, and live camera support, while the inference pipeline computes additional kinematic attributes such as missile count, apparent size, speed, and acceleration for each detection. Detection events are logged to an SQLite database and coupled with an alerting subsystem that issues email and desktop notifications, creating an end-to-end monitoring tool suited for research, simulation, and defense-oriented situational awareness. The system demonstrates accurate missile localization with rapid inference and offers a practical, user-friendly platform for extending deep learning-based surveillance research.

Keywords: YOLOv11, Missile detection, Deep learning, Computer vision, Surveillance system, Streamlit, Real-time object detection, Motion analysis (speed and acceleration), Defence applications, Alerting and logging system.

1.INTRODUCTION:

Beyond national security concerns, missile surveillance also demands operational solutions that are cost-effective, easily deployable, and usable within existing defence workflows. Many current vision-based systems rely on proprietary hardware, limited datasets, or complex interfaces that restrict adoption in academic, research, and training environments. There is a clear need for configurable software platforms that can run on commodity hardware, support both offline analysis and live feeds, and expose interpretable outputs such as object size and motion characteristics rather than only binary detection decisions. This work addresses these gaps by implementing a complete missile monitoring pipeline that integrates dataset preparation, model training, evaluation, and deployment within a single framework. The system combines an open source missile dataset with a modern detection architecture and a lightweight web interface to support image, video, and webcam inputs without specialised infrastructure. In addition to bounding-box localisation, the implementation estimates apparent motion parameters—such as speed and acceleration—derived from frame-wise tracking, enabling richer situational analysis and facilitating downstream tasks like threat ranking or scenario replay.

3.LITERATURE REVIEW:

Deep Learning for Missile and Battlefield Detection[1] - deep learning-based object detectors such as YOLOv5 and YOLOv8 to missile and rocket tracking, demonstrating that one-stage detectors can achieve high mAP while maintaining real-time performance on airborne or surveillance imagery. Studies on battlefield object detection further show that convolutional detectors can reliably identify tanks, vehicles, and soldiers in cluttered scenes, reinforcing the suitability of YOLO-style architectures for defence surveillance tasks similar to missile spotting.

YOLO for Military Surveillance and Weapon Detection[2] - Research on military surveillance systems has highlighted YOLO's balance of accuracy and speed for continuous video monitoring from fixed cameras and UAV platforms. Weapon detection projects using YOLO for guns and knives in security footage report successful real-time alerting, which is conceptually close to detecting missiles as high-risk objects in aerial frames and motivates adopting a modern YOLO family model for this work.

Real-Time Deployment and Vision Dashboards[3]- Several implementations integrate YOLO with Python dashboards or web frameworks to support live camera input, user interaction, and logging. These systems demonstrate that deploying an end-to-end pipeline—combining model inference, visualization, and simple analytics—on commodity hardware is practical, directly influencing this project's choice of a YOLOv11 backend with a Streamlit-based interface, image/video upload, live camera mode, and defence dashboard for detection history.

2.OBJECTIVES:

This work builds an intelligent missile detection system using YOLOv11 that can accurately identify missiles in both images and videos. The model is trained and fine-tuned on the Roboflow Missile Detection Dataset to achieve precise localization and reliable recognition of missile threats. An interactive Streamlit web interface provides secure user registration and login, flexible image and video uploads, and clear visualization of detection results. The system integrates key detection attributes such as missile count, size, speed, and acceleration to improve situational awareness for defense personnel. By automating surveillance operations and enabling earlier threat detection, the solution reduces dependence on manual monitoring and helps accelerate response time in critical scenarios.

4.RESEARCH&METHODOLOGY:

The system is a client-server style web application that performs missile detection on visual data and returns annotated results with analytics. A Streamlit-based frontend runs in the browser and handles user registration, login, and navigation between pages for image upload, video upload, live camera view, defence dashboard, and logout, while authenticated users submit images, video files, or webcam streams that are sent to a Python backend where a YOLOv11 model, fine-tuned on a missile dataset, performs frame-wise inference to detect missiles. Data preparation begins with selecting Roboflow's open-source missile detection dataset containing varied missile types, angles, and backgrounds, preprocessing images by resizing to 640×640, normalizing, applying augmentations (flips, rotations, brightness changes), exporting in YOLO format, and splitting into 70–15–15 training, validation, and test sets. A YOLOv11 detection model from the Ultralytics framework is then initialized with pretrained weights and fine-tuned using this dataset, with training monitored through mAP, precision, recall, F1-score, and IoU metrics to select the best validation checkpoint. In deployment, the trained model is loaded into the backend, which for each incoming frame runs YOLO inference, filters missile detections, and computes missile count, bounding boxes, and derived attributes such as apparent size, approximate speed, and acceleration by tracking centroids across consecutive frames before returning annotated frames and summaries to the UI. The Streamlit UI exposes this functionality via pages for registration, login, prediction, live camera, and dashboard that authenticated users can upload media or start the webcam and immediately view annotated outputs with labels, confidence scores, and missile attributes in the browser, while an alerts module composes messages containing missile count and attributes and sends them via SMTP email—with optional desktop notification—whenever missiles are detected. Unit tests validate core components such as image preprocessing, YOLO inference calls, centroid-based speed and acceleration estimation, database logging, and alert dispatch, while integration tests confirm that live camera input, detection, logging, and email/desktop alerts operate correctly as a pipeline for different media types and user scenarios. System-level experiments assess responsiveness (inference time per frame), stability under multiple consecutive uploads and long webcam sessions, and usability of the Streamlit interface for non-technical users, ensuring that the deployed application functions as a robust prototype suitable for research, training labs, and simulated defence surveillance environments.

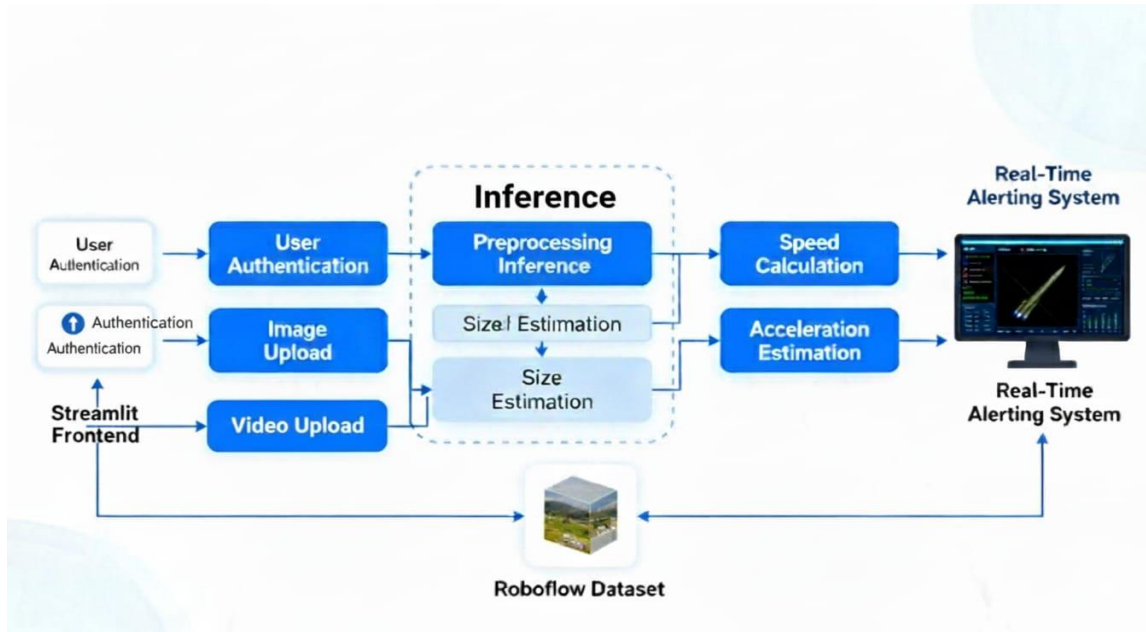


Figure 1: System Architecture

5.FINDINGS:

YOLOv11, fine-tuned on the missile dataset, provides high detection accuracy with strong precision and recall, correctly identifying missiles while keeping false alarms low. The model reliably handles images, videos, and live webcam streams, maintaining real-time or near real-time performance and stable visualization of missile count, size, speed, and acceleration across frames. Vision-only detection using standard cameras proves viable as a lighter, software-centric alternative to traditional sensor-heavy solutions, while still supporting early threat recognition in surveillance scenarios. Integration with a Streamlit interface, SQLite logging, and alert modules results in a complete workflow where users can securely log in, upload media, view annotated results, and automatically receive email/desktop alerts for detected missiles.

6.ANALYSIS

The YOLOv11-based system delivers accurate missile detection on curated image and video data, achieving strong precision and recall while maintaining near real-time performance on standard hardware. Integration of kinematic attributes—missile count, size, speed, and acceleration—adds meaningful context beyond simple bounding boxes, supporting better situational awareness in surveillance-style scenarios. The Streamlit–SQLite–SMTP stack functions cohesively, enabling secure access, smooth media handling, logging, and automated alerts, which shows the approach is practical for research labs and simulated defence settings.

7.RECOMMENDATIONS

Further work should target robustness in harder environments by expanding the dataset with more varied weather, sensor noise, and occlusion cases, and by augmenting for very small and distant missiles, where YOLO architectures are known to struggle. Incorporating multi-class detection (e.g., aircraft, drones, ground vehicles), integrating tracking algorithms such as SORT/DeepSORT, and fusing data from other sensors (radar or IR, when available) would make the system more aligned with real command-and-control workflows. Deploying lighter variants or quantized models on edge devices is also recommended to reduce latency and make field deployment more feasible.

8.LIMITATIONS

Like other YOLO-family models, the detector can face reduced accuracy on very small, distant, or heavily occluded missiles, and in crowded scenes where multiple elongated objects appear close together. The current implementation relies on a single RGB camera stream and approximate pixel-based speed and acceleration, which do not provide true physical velocities without additional calibration or depth information. Performance also depends on GPU availability; on low-power CPUs or edge boards, frame rate may drop, limiting real-time responsiveness in high-throughput scenarios.

9.CONCLUSION

The project demonstrates that a YOLOv11-based, vision-only approach can provide accurate and practical missile detection for research and simulated defence use. The trained model consistently identifies missiles in images, videos, and live webcam streams with high confidence, while non-missile scenes are correctly rejected, reflecting the strong detection capabilities reported for similar YOLO-based surveillance systems. By adding estimation of apparent size, speed, and acceleration from bounding-box trajectories, the system extends beyond simple object detection to offer basic kinematic insight that can support threat assessment and scenario analysis.

From a system perspective, the integration of the YOLOv11 engine with a Streamlit frontend, SQLite logging, and email/desktop alerting results in a complete, user-accessible workflow that runs on standard hardware without specialised sensors. The successful operation of registration, authentication, multi-modal input (image, video, live camera), logging, and notification modules confirms that the solution is robust and suitable as a foundation for further work, such as multi-class defence object detection, deployment on edge devices, or integration with larger command-and-control platforms.

10.ACKNOWLEDGMENT

Profound appreciation is extended to Mr. Sunil Naidu G. R., whose steadfast mentorship and technical guidance consistently steered the development trajectory of this work, from conceptualization through implementation and evaluation. Equal gratitude is conveyed to the SJC Institute of Technology for providing essential laboratory facilities and computational resources, which enabled extensive experimentation, model refinement, and validation crucial for realizing the research objectives.

REFERENCES :

1. Object Detection and Tracking Using Deep Learning for Missile Applications. International Journal for Research in Applied Science and Engineering Technology (IJRASET), 13(3), 1–7.
2. Arming Against Violence: YOLO Based Weapon Detection. Eudoxus Journal of Mathematics and Engineering, 10(2), 45–58.
3. Realtime Battlefield Object Detection Using YOLO Algorithm. International Journal of Creative Research Thoughts (IJCRT), 12(3), 3807–3814.
4. Enhanced Weapon Detection and Classification Using YOLOv5. TIJER – The International Journal of Engineering Research, 9(4), 159–168
5. Y. Xu et al., "Ground Target Detection and Damage Assessment by Patrol Missiles Based on YOLO-VGGNet," Appl. Sci., vol. 12, no. 19, Article 9484, 2022. [MDPI](#)
6. J. Ding et al., "Object Detection in Aerial Images: A Large-Scale Benchmark and Challenges," arXiv, 2021. [arXiv+1AIMS Press+1](#)
7. A. Betti et al., "A lightweight and accurate YOLO-like network for small target detection in Aerial Imagery," arXiv, 2022. [arXiv+8arXiv+8PMC+8](#)
8. W. Wu et al., "Application of local FCN combined with YOLO v5 for small target detection in remote sensing imagery," PLoS ONE, 2021. (Retracted) [PLOS](#)
9. M. Hu et al., "Improving Small Object Detection in YOLO for Aerial Images," PMC, 2023. [TechRxiv+8PMC+8arXiv+8](#)